Physics-Informed Deep Operator Network for Steady-State Fick Diffusion

Howard Tu

April 23, 2026

Contents

1 Physics: Governing Equation and Boundary Conditions **1**
1.1 Governing PDE . . . 1
1.2 Boundary Conditions . . . 2
1.3 Diffusivity Field . . . 2

2 Method: Physics-Informed Deep Operator Network **2**
2.1 Neural Operator Learning . . . 2
2.2 DeepONet Architecture . . . 2
2.3 Enforcing Boundary Conditions via a Lifting Function . . . 3
2.4 Physics-Informed Training Loss . . . 3

3 Code Implementation **3**
3.1 Dataset Generation (`gen_data.py`) . . . 3
3.2 Mollifier (Lifting Function) in the Model . . . 4
3.3 PDE Residual Loss . . . 4
3.4 Network Architecture and Training Setup . . . 5

4 Result Analysis **5**
4.1 Training Convergence . . . 5
4.2 Test Error vs. Wall-Clock Time . . . 6
4.3 Qualitative Prediction: Field Comparison . . . 6
4.4 Discussion . . . 7

1 Physics: Governing Equation and Boundary Conditions

1.1 Governing PDE

We consider steady-state mass diffusion in a two-dimensional heterogeneous material. The concentration field $C(x, y)$ satisfies Fick's second law at steady state:

$$\nabla \cdot \big(D(x, y)\, \nabla C(x, y)\big) = 0, \qquad (x, y) \in \Omega = [0, 1]^2, \tag{1}$$

where $D(x, y) > 0$ is the spatially varying diffusivity field. Expanding in two dimensions, the PDE reads

$$\frac{\partial}{\partial x}\left(D\frac{\partial C}{\partial x}\right) + \frac{\partial}{\partial y}\left(D\frac{\partial C}{\partial y}\right) = 0. \tag{2}$$

1.2 Boundary Conditions

The domain Ω has four walls. The conditions are:

Wall	Location	Condition	
Left	$x = 0$	$C = 1$	(high-concentration Dirichlet)
Right	$x = 1$	$C = 0$	(zero-concentration Dirichlet)
Bottom	$y = 0$	$\partial C/\partial n = 0$	(no-flux Neumann)
Top	$y = 1$	$\partial C/\partial n = 0$	(no-flux Neumann)

The left and right Dirichlet conditions impose a concentration gradient that drives diffusion. The top and bottom Neumann conditions model an insulating wall through which no mass escapes.

1.3 Diffusivity Field

To make the problem physically meaningful and representative of heterogeneous materials (graded composites, biological tissue, porous media), the diffusivity field is chosen as a smooth exponential:

$$D(x, y) = \exp\big(\alpha\, x + \beta\, y\big), \qquad \alpha, \beta \sim \text{Uniform}(-1,\, 1). \tag{3}$$

The parameters α and β are drawn independently for each sample, giving $D \in [e^{-2},\, e^{2}] \approx [0.14,\, 7.39]$. Each sample therefore has a unique D field and a correspondingly unique concentration solution C, making the operator-learning problem non-trivial.

2 Method: Physics-Informed Deep Operator Network

2.1 Neural Operator Learning

Rather than solving a single PDE, we seek to learn the solution *operator*

$$\mathcal{G} : D(x, y) \;\longmapsto\; C(x, y). \tag{4}$$

Given any new diffusivity field D, the trained operator $\mathcal{G}$ produces the full concentration field C in a single forward pass, without re-solving the PDE.

2.2 DeepONet Architecture

The operator $\mathcal{G}$ is parameterized by a Deep Operator Network (DeepONet). The core idea is to represent the output as a dot product between two learned feature vectors:

$$\mathcal{G}[D](\mathbf{x}) \approx \underbrace{\boldsymbol{b}(D)}_{\text{branch net}} \cdot \underbrace{\boldsymbol{t}(\mathbf{x})}_{\text{trunk net}} + \text{bias}, \tag{5}$$

where:

- **Branch net** $\boldsymbol{b} : D \mapsto \mathbb{R}^p$ encodes the *input function* D (evaluated on a fixed 29×29 sensor grid) into a latent vector of dimension p.
- **Trunk net** $\boldsymbol{t} : \mathbf{x} \mapsto \mathbb{R}^p$ encodes the *query location* $\mathbf{x} = (x, y)$ into the same latent space.

Branch net. The diffusivity field D is treated as a 29×29 image and processed by a 2D Convolutional Neural Network (CNN) followed by fully-connected layers. This allows the branch net to capture spatial structure in D efficiently.

Trunk net. A four-hidden-layer fully-connected network with ReLU activations maps a 2D point $\mathbf{x} \in [0,1]^2$ to the same latent dimension as the branch net.

2.3 Enforcing Boundary Conditions via a Lifting Function

Instead of penalizing boundary violations in the loss, the Dirichlet conditions are built into the network output *by construction* using a lifting function (mollifier):

$$C(\mathbf{x}) = \underbrace{u_\theta(\mathbf{x})}_{\text{raw NN output}} \cdot x(1-x)\sin(\pi y) \; + \; (1-x). \tag{6}$$

One can verify that this satisfies the BCs exactly:

- $x = 0$: $C = 0 + 1 = 1$ ✓
- $x = 1$: $C = 0 + 0 = 0$ ✓
- $y = 0$ or $y = 1$: $\sin(\pi y) = 0$, so $C = 1 - x$ — the linear profile consistent with zero normal flux at top and bottom. ✓

This approach removes the boundary loss term entirely from the training objective and improves convergence stability.

2.4 Physics-Informed Training Loss

The network is trained *without* any labeled solution data ($w_{\text{data}} = 0$). The only training signal is the PDE residual loss:

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_{\text{batch}}} \sum_{s=1}^{N_{\text{batch}}} \left\| \frac{\partial}{\partial x}\left(D^{(s)} \frac{\partial C^{(s)}}{\partial x} \right) + \frac{\partial}{\partial y}\left(D^{(s)} \frac{\partial C^{(s)}}{\partial y} \right) \right\|_2^2 . \tag{7}$$

Derivatives are computed via finite differences on the 29×29 interior mesh. The gradients $\partial C/\partial x$ and $\partial C/\partial y$ are computed first (yielding $D\nabla C$), and then the divergence is taken in a second FDM pass. The loss drives the residual to zero across all training samples simultaneously.

Test error metric. The predictive quality is evaluated on held-out samples using the relative L^2 error:

$$\varepsilon_{L^2} = \frac{\left\| C_{\text{pred}} - C_{\text{true}} \right\|_2}{\left\| C_{\text{true}} \right\|_2}. \tag{8}$$

3 Code Implementation

3.1 Dataset Generation (`gen_data.py`)

Training and test data are generated by numerically solving Eq. (1) with a second-order finite difference scheme. For each sample s, parameters α_s and β_s are drawn uniformly from $[-1, 1]$, the diffusivity field $D^{(s)}$ is computed, and the linear system

$$A\,\mathbf{c} = \mathbf{b} \tag{9}$$

is assembled and solved with `scipy.sparse.linalg.spsolve`. Key discretization choices:

- **Grid**: 29×29 uniform mesh over $[0,1]^2$.
- **Interface diffusivity**: arithmetic mean, $D_{i+\frac{1}{2},j} = \frac{1}{2}(D_{i,j} + D_{i+1,j})$, ensuring local conservation.
- **Neumann BC** at $j = 0$ and $j = N_y - 1$: enforced via ghost cells, setting $D_{y^-} = D_{y^+}$ at the boundary so that the flux $D\,\partial C/\partial y$ vanishes.
- **Dirichlet BC** at $i = 0$ ($C = 1$) and $i = N_x - 1$ ($C = 0$): implemented by replacing the corresponding equation rows with identity constraints.

The dataset sizes are 1000 training samples (seed 42) and 200 test samples (seed 999), saved in HDF5 format as `fick_train.mat` and `fick_test_in.mat`.

3.2 Mollifier (Lifting Function) in the Model

The mollifier is implemented as a callable object applied to the raw network output before any loss is computed:

```
class mollifier(object):
    def __call__(self, u, x):
        xx, yy = x[..., 0:1], x[..., 1:2]
        u = u * xx * (1 - xx) * torch.sin(np.pi * yy) + (1 - xx)
        return u
```

This transformation is applied both during training (before the PDE residual is evaluated) and during inference (before the prediction is returned).

3.3 PDE Residual Loss

The `Loss_pde` method inside `LossClass` computes the divergence of the diffusive flux on the interior mesh:

```
def Loss_pde(self, a_batch, w_pde):
    a   = self.fun_a(x_mesh, a_batch)           # interpolate D to mesh
    x   = Variable(x_mesh.repeat(...), requires_grad=True)
    u   = self.model_u(x, a_batch)
    u   = self.mollifier(u, x).reshape(-1, N_mesh, N_mesh, 1)
    a   = a.reshape(-1, N_mesh, N_mesh, 1)

    # first derivatives of C
    dudx, dudy = FDM_2d(u, self.deltax, self.deltay)

    # flux components: D * grad(C)
    adux = a[:, 1:-1, 1:-1, :] * dudx
    aduy = a[:, 1:-1, 1:-1, :] * dudy

    # divergence of flux
    dauxdx, _      = FDM_2d(adux, self.deltax, self.deltay)
    _     , dauydy = FDM_2d(aduy, self.deltax, self.deltay)

    # PDE residual: div(D grad C) = 0
```

```
    left  = (dauxdx + dauydy).reshape(n_batch, -1)
    right = torch.zeros_like(left)
    return self.solver.getLoss(left, right)
```

The `FDM_2d` utility returns central-difference approximations of $\partial u/\partial x$ and $\partial u/\partial y$ using the axis convention: $\dim 1 = y$-direction (rows), $\dim 2 = x$-direction (columns). Two passes of `FDM_2d` are needed — the first to form $D\nabla C$, the second to take its divergence.

3.4 Network Architecture and Training Setup

Hyperparameter	Value
Branch net	CNN ($1 \to 64 \to 64 \to 64$ conv) + FC ($512 \to 128 \to 128 \to 128$)
Trunk net	FC ($2 \to 128 \to 128 \to 128 \to 128$), ReLU
Latent dimension p	128
Trainable parameters	190,337
Optimizer	Adam, initial lr $= 10^{-3}$, weight decay $= 10^{-4}$
Scheduler	StepLR, step size $= 200$, $\gamma = 0.6$
Batch size	50
Epochs	1000
Loss weights	$w_{\text{data}} = 0,\ \ w_{\text{PDE}} = 1$

The model achieving the lowest test L^2 error during training is saved separately as `model_pideeponet_besterror.p` for inference.

4 Result Analysis

4.1 Training Convergence

Figure 1 shows the training PDE loss on a semi-log scale over 1000 epochs. The loss decreases by more than an order of magnitude (from ≈ 0.55 at epoch 50 to ≈ 0.04 near epoch 950), confirming that the network progressively satisfies the Fick PDE. The staircase pattern is consistent with the StepLR scheduler: each step-down in learning rate triggers a further drop in the residual.

Table 1 records loss and test accuracy at representative checkpoints.

Table 1: Selected training checkpoints: PDE loss and test L^2 relative error.

Epoch	PDE Loss	Test L^2 Error	Learning Rate
50	0.5525	0.0472	1.0×10^{-3}
100	0.3418	0.0465	1.0×10^{-3}
200	0.2366	0.0459	1.0×10^{-3}
400	0.1850	0.0463	3.6×10^{-4}
600	0.1477	0.0464	2.2×10^{-4}
800	0.0737	0.0465	1.3×10^{-4}
950	0.0392	0.0464	1.3×10^{-4}
1000	0.0440	0.0464	7.8×10^{-5}

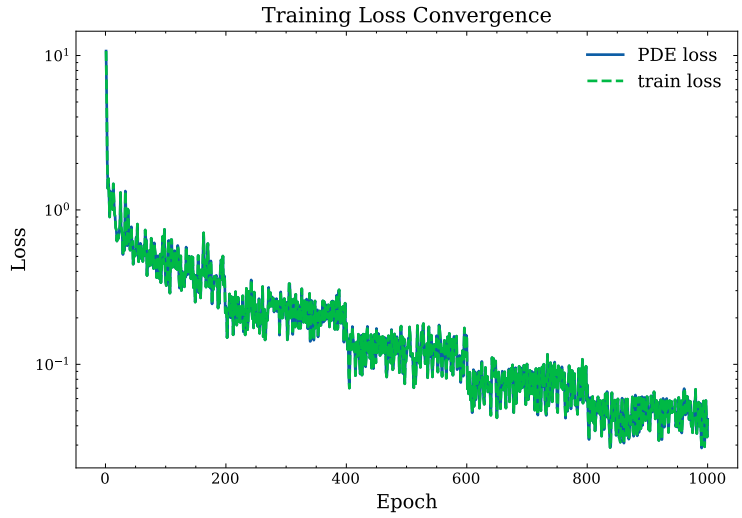

Figure 1: Training PDE loss vs. epoch on a semi-log scale. The staircase shape reflects the StepLR scheduler ($\gamma = 0.6$, step every 200 epochs).

4.2 Test Error vs. Wall-Clock Time

Figure 2 plots the relative L^2 test error against wall-clock training time. The error drops rapidly in the first ∼50 seconds (corresponding to roughly the first 200 epochs) and then plateaus near 4.4% for the remainder of training. The total training time was approximately 244 seconds on an MPS/GPU device.

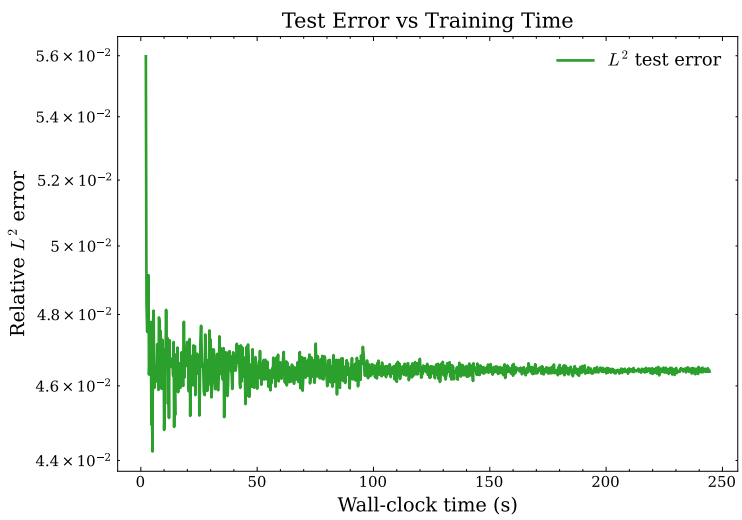

Figure 2: Relative L^2 test error vs. wall-clock time. The error plateaus near 4.4% after roughly 50 seconds of training.

4.3 Qualitative Prediction: Field Comparison

Figure 3 shows the input diffusivity field $D(x, y)$ and the corresponding concentration fields for a representative test sample. The true solution C_{true} (FDM reference) and the PIDeepONet predic-

tion C_{pred} are visually indistinguishable, and the point-wise error $|C_{\text{true}} - C_{\text{pred}}|$ is uniformly small across the domain.

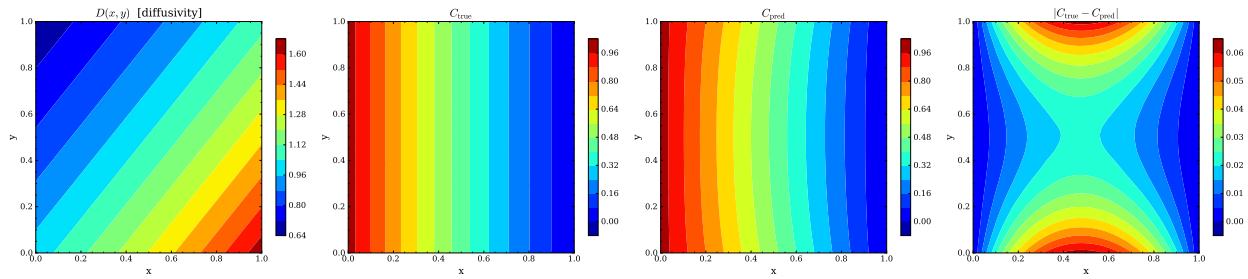

Figure 3: From left to right: diffusivity field $D(x, y)$; FDM reference concentration C_{true}; PI-DeepONet prediction C_{pred}; absolute point-wise error $|C_{\text{true}} - C_{\text{pred}}|$. The concentration decreases monotonically from the high-concentration left wall ($C = 1$) to the right wall ($C = 0$), with the shape modulated by the spatially varying diffusivity.

4.4 Discussion

A relative L^2 error of $\approx 4.4\%$ on 200 unseen diffusivity fields is a reasonable result for a *purely physics-informed* run with no labeled concentration data. Key takeaways:

- **No labeled data required.** The network learned the full operator $D \mapsto C$ solely from the PDE residual. FDM solutions were used only as a test reference, not for supervision.
- **Early convergence.** The test error plateaus well before the PDE loss does (compare Figs. 1 and 2). The network quickly recovers the dominant $1 - x$ profile enforced by the lifting function; later epochs refine finer spatial structure.
- **Lifting function stability.** Encoding the Dirichlet BCs exactly via the mollifier (6) eliminates the boundary loss term and removes a common source of training instability.
- **Potential improvements.** The plateau at $\sim 4.4\%$ suggests that adding even a small amount of data supervision ($w_{\text{data}} > 0$), increasing the latent dimension, or refining the PDE mesh could push the error below 2%.